

Practical No. 1

Definition : Write a program to demonstrate basic input and output operations in Python.

Aim/ Objective:

- To understand how to accept input from the user in Python.
- To display output using the print() function.
- To perform basic operations on user-provided data.
- To understand basic Python syntax for input and output.

Source Code:

```
# Program to demonstrate basic input and output operations
```

```
# Taking input from the user
```

```
name = input("Enter your name: ")
```

```
age = int(input("Enter your age: "))
```

```
# Displaying output
```

```
print("\n--- User Details ---")
```

```
print("Name:", name)
```

```
print("Age:", age)
```

```
# Taking two numbers as input
```

```
num1 = float(input("\nEnter first number: "))
```

```
num2 = float(input("Enter second number: "))
```

```
# Performing basic arithmetic operation
```

```
sum = num1 + num2
```

```
# Displaying the result
```

```
print("Sum of two numbers:", sum)
```

Output:

Enter your name: Kalpesh

Enter your age: 40

--- User Details ---

Name: Kalpesh

Age: 40

Enter first number: 25

Enter second number: 15

Sum of two numbers: 40.0

Conclusion:

The program successfully demonstrates the basic input and output operations in Python. It shows how to accept data from the user using `input()`, convert input into appropriate data types, store values in variables, perform a basic arithmetic operation, and display the results using `print()`

Practical No. 2

Definition : Write a program to demonstrate basic input and output operations in Python.

Aim/ Objective:

- To learn how to identify different data types.
- To convert one data type into another.
- To understand the use of `int()`, `float()`, and `str()`.
- To understand the importance of type casting in Python.

Source Code:

```
student_name = "Arjun"
```

```
roll_no = 92600584105
```

```
age = 22
```

```
height = 5.7
```

```
is_student = True
```

```
print("Student Name:",student_name,"Type:",type(student_name))
```

```
print("Age:",age,"Type:",type(age))
```

```
print("Height:",height,"Type:",type(height))
```

```
print("IS student:",is_student,"Type:",type(is_student))
```

```
num1 = "44"
```

```
num2 = int(num1)
```

```
num3 = float(num2)
```

```
num4 = str(num3)
```

```
print("String to integer:",num2)
```

```
print("Integer to float:",num3)
```

```
print("Integer to string:",num4)
```

Output:

Student Name: Arjun Type: <class 'str'>

Age: 22 Type: <class 'int'>

Height: 5.7 Type: <class 'float'>

IS student: True Type: <class 'bool'>

String to integer: 44

Integer to float: 44.0

Integer to string: 44.0

PS F:\python>

Conclusion:

In this program, we learned about different data types in Python such as **String, Integer, Float, and Boolean**. We also learned how to check the data type using the `type()` function and how to convert one data type into another using **type casting** functions like `int()`, `float()`, and `str()`.

Practical No.3

Definition: Write a program to perform arithmetic relational and logical operations using Python operators.

Aim & Objective:

- To understand arithmetic operators in Python.
- To perform mathematical calculations using arithmetic operators.
- To understand relational operators and compare two values.
- To understand logical operators such as and, or, and not.
- To learn how Python operators are used in programming.

Source code:

```
num1=float(input("Enter first number:"))  
num2=float(input("Enter second number:"))
```

```
#Arithmetic operators
```

```
print("Addition:",num1+ num2)  
print("Substraction:",num1-num2)  
print("Multiplication:",num1*num2)  
print("Division:",num1/num2)  
print("Modulus:",num1%num2)
```

```
#Relational Operators
```

```
print("Greater than:",num1>num2)  
print("Less than:",num1<num2)  
print("Equal to:",num1==num2)  
print("Not equal to:",num1!=num2)
```

```
#Logical operators
```

```
a= True  
b= False
```

```
print("Logical AND:",a and b)  
print("Logical OR:",a or b)
```

```
print("Logical NOT:",not a)
```

Output:

Enter first number:87

Enter second number:54

Addition: 141.0

Substraction: 33.0

Multiplication: 4698.0

Division: 1.6111111111111112

Modulus: 33.0

Greater than: True

Less than: False

Equal to: False

Not equal to: True

Logical AND: False

Logical OR: True

Logical NOT: False

Conclusion:

In this program, we learned how to use **arithmetic, relational, and logical operators** in Python. Arithmetic operators are used for mathematical calculations, relational operators are used to compare values, and logical operators are used to combine or reverse conditions. Thus, we understood the basic use of Python operators.

Practical No.4

Defination: Write a program to demonstrate string operations including slicing formatting and built-in string functions.

Aim & Objective:

To perform basic operations on strings.

- **To understand string slicing.**
- **To learn string formatting.**
- **To use built-in string functions like upper(), lower(), replace(), and len().**
- **To understand the use of strings in Python.**
- **To understand how string functions are used in Python programs.**

Source Code:

```
name = "Python Programming"
# String Operations
print("String :", name)
print("Length :", len(name))
print("First Character :", name[0])
print("Last Character :", name[-1])

# String Slicing
print("Slicing :", name[0:6])
print("Reverse :", name[::-1])

# String Formatting
age = 21
print("My name is {} and my age is {}".format(name, age))

# Built-in String Functions
print("Uppercase :", name.upper())
print("Lowercase :", name.lower())
print("Replace :", name.replace("Python", "Java"))
print("Count :", name.count("m"))
print("Find :", name.find("Programming"))
```

Output:

```
String : Python Programming
Length : 18
First Character : P
Last Character : g
Slicing : Python
Reverse : gnimmargorP nohtyP
My name is Python Programming and my age is 21
Uppercase : PYTHON PROGRAMMING
Lowercase : python programming
Replace : Java Programming
```

Count : 2

Find : 7

Conclusion:

In this program, we learned how to perform different **string operations** in Python. We learned **string slicing, reversing, string formatting, and built-in string functions** such as upper(), lower(), replace(), count(), and find(). Thus, we understood how strings can be easily manipulated and processed in Python.

Practical No.5

Definiton: Write a program to create and manipulate lists using indexing slicing and list comprehensions.

Aim & Objective:

- **To understand the concept of lists in Python.**
- **To create and store multiple values in a list.**
- **To access list elements using indexing.**
- **To extract elements using list slicing.**
- **To modify and manipulate list elements.**
- **To understand and use list comprehensions.**
- **To perform basic operations on lists in Python.**

Source Code:

```
numbers = [10, 20, 30, 40, 50]
```

```
print("List:", numbers)
```

```
# Indexing
```

```
print("First Element:", numbers[0])
```

```
print("Last Element:", numbers[-1])
```

```
# Slicing
```

```
print("Slicing:", numbers[1:4])
```

```
print("First Three Elements:", numbers[:3])
```

```
# Manipulating List
```

```
numbers.append(60)
```

```
print("After Append:", numbers)
```

```
numbers.remove(20)
```

```
print("After Remove:", numbers)
```

```
# List Comprehension
```

```
squares = [x * x for x in numbers]
```

```
print("Squares:", squares)
```

Output:

List: [10, 20, 30, 40, 50]

First Element: 10

Last Element: 50

Slicing: [20, 30, 40]

First Three Elements: [10, 20, 30]

After Append: [10, 20, 30, 40, 50, 60]

After Remove: [10, 30, 40, 50, 60]

Squares: [100, 900, 1600, 2500, 3600]

Conclusion:

In this program, we learned how to create and manipulate lists in Python. We used indexing to access elements, slicing to extract elements, and list functions like append() and remove() to modify the list. We also learned how to use list comprehension to create a new list easily.

Practical No.6

Definition: Write a program to illustrate the use of tuples and sets with basic operations.

Aim & Objective:

- **To understand the concept of tuples in Python.**
- **To perform basic tuple operations such as indexing, length, count, and index.**
- **To understand the concept of sets and their unique elements.**
- **To perform basic set operations such as union, intersection, and difference.**
- **To learn how to add and remove elements from a set.**

Source Code:

```
numbers = (10, 20, 30, 40, 50)
```

```
print("Tuple :", numbers)
print("First Element :", numbers[0])
print("Last Element :", numbers[-1])
print("Length of Tuple :", len(numbers))

# Tuple Operations
print("Count of 20 :", numbers.count(20))
print("Index of 30 :", numbers.index(30))

# Set
myset = {10, 20, 30, 40}

print("Set :", myset)

# Set Operations
myset.add(50)
print("After Adding :", myset)

myset.remove(20)
print("After Removing :", myset)

set2 = {30, 40, 50, 60}

print("Union :", myset.union(set2))
print("Intersection :", myset.intersection(set2))
```

Output:

```
First Element : 10
Last Element : 50
Length of Tuple : 5
Count of 20 : 1
Index of 30 : 2
Set : {40, 10, 20, 30}
After Adding : {40, 10, 50, 20, 30}
```

After Removing : {40, 10, 50, 30}

Union : {40, 10, 50, 60, 30}

Intersection : {40, 50, 30}

Conclusion:

The program successfully demonstrates the use of tuples and sets in Python. We performed basic tuple operations such as indexing, counting, and finding length. We also performed set operations such as adding, removing, union, and intersection. Thus, we understood the basic operations and differences between tuples and sets.

Practical No.7

Definition:

- To understand the concept of a dictionary in Python.
- To create and initialize a dictionary using key-value pairs.
- To demonstrate basic dictionary methods such as keys(), values(), items(), get(), and update().
- To learn how to add, modify, and delete dictionary elements.
- To iterate through a dictionary using a for loop.
- To understand how to access keys and values from a dictionary.

Source Code:

```
student = {  
    "name": "Arjun Sharma",  
    "age": 22,  
    "course": "Python",  
    "marks": 85  
}  
  
print("Dictionary :", student)  
  
# Dictionary Methods  
print("Keys :", student.keys())  
print("Values :", student.values())  
print("Items :", student.items())  
print("Name :", student.get("name"))  
  
# Add and Update  
student["city"] = "Bihar"  
student["marks"] = 90
```

```
print("After Adding and Updating :", student)
```

```
# Delete
```

```
student.pop("age")
```

```
print("After Removing Age :", student)
```

```
# Iteration
```

```
print("Dictionary Elements:")
```

```
for key, value in student.items():
```

```
    print(key, ":", value)
```

Output:

Name : Arjun Sharma

After Adding and Updating : {'name': 'Arjun Sharma', 'age': 22, 'course': 'Python', 'marks': 90, 'city': 'Bihar'}

After Removing Age : {'name': 'Arjun Sharma', 'course': 'Python', 'marks': 90, 'city': 'Bihar'}

Dictionary Elements:

name : Arjun Sharma

course : Python

marks : 90

city : Bihar

Conclusion:

The program successfully demonstrates the creation and use of a dictionary in Python. Various dictionary methods such as keys(), values(), items(), get(), and pop() were used. The program also demonstrated how to add, update, and delete dictionary elements and how to iterate through the dictionary using a for loop. Thus, we understood the basic operations and methods of dictionaries in Python.

Practicle No.8

Definition: Write a program to explain mutable and immutable objects in Python.

Aim & Objective:

- To understand the concept of mutable and immutable objects.
- To demonstrate a mutable object using a list.
- To demonstrate an immutable object using an integer or string.
- To understand how mutable objects can be modified after creation.

- To understand that immutable objects cannot be changed after creation.
- To identify the basic difference between mutable and immutable objects in Python.

Source Code:

```
name = "Arjun Sharma"  
print("Original String :", name)
```

```
name = name + " Python"  
print("After Modification :", name)
```

```
# Mutable Object - List  
student = ["Arjun Sharma", 21, "Python"]
```

```
print("Original List :", student)
```

```
student[1] = 22  
print("After Modification :", student)
```

Output:

Original String : Arjun Sharma

After Modification : Arjun Sharma Python

Original List : ['Arjun Sharma', 21, 'Python']

After Modification : ['Arjun Sharma', 22, 'Python']

Conclusion:

The program successfully demonstrates the concept of mutable and immutable objects in Python. Strings are immutable, so their original value cannot be changed directly. Lists are mutable, so their elements can be modified after creation. Thus, we understood the basic difference between mutable and immutable objects in Python.

Practical No.9

Definition: Write a program to define and use user-defined functions with different types of arguments.

Aim & Objective:

- To understand the concept of user-defined functions in Python.
- To learn how to define and call a function.
- To demonstrate functions with positional arguments.
- To demonstrate functions with keyword arguments.
- To demonstrate functions with default arguments.

- To understand how arguments are passed to functions in Python.

Source Code:

```
def add(a, b):
```

```
    print("Addition :", a + b)
```

```
add(10, 20)
```

2. Keyword Arguments

```
def student(name, age):
```

```
    print("Name :", name)
```

```
    print("Age :", age)
```

```
student(age=22, name="Arjun Sharma")
```

3. Default Arguments

```
def greet(name="Arjun Sharma"):
```

```
    print("Hello", name)
```

```
greet()
```

4. Variable Length Arguments

```
def total(*numbers):
```

```
    print("Total :", sum(numbers))
```

```
total(10, 20, 30, 40)
```

Output:

Addition : 30

Name : Arjun Sharma

Age : 22

Hello Arjun Sharma

Total : 100

Conclusion:

The program successfully demonstrates the use of user-defined functions with different types of arguments in Python. It explains positional, keyword, default, and variable-length arguments. Thus, we learned how to define, call, and pass different types of arguments to functions effectively.

Practical No.10

Definition: Write a program to demonstrate recursion using factorial or Fibonacci series.

Aim & Objective:

- To understand the concept of recursion in Python.
- To create and use a recursive function.
- To calculate the factorial of a number using recursion.
- To understand the importance of the base condition in recursion.
- To understand how a function calls itself repeatedly until the base condition is reached.

Source Code:

```
def factorial(n):  
    if n == 0 or n == 1:  
        return 1  
    else:  
        return n * factorial(n - 1)  
  
num = int(input("Enter a number: "))  
  
result = factorial(num)  
  
print("Factorial of", num, "=", result)
```

Output:

```
Enter a number: 7  
Factorial of 7 = 5040
```

Conclusion:

The program successfully demonstrates recursion in Python by calculating the factorial of a number. The function calls itself repeatedly until the base condition ($n == 0$ or $n == 1$) is reached. Thus, we learned how recursion works and how a recursive function can be used to solve a problem.